

Contents

Statistical Analysis Report	1
Notebook-Based EDA: Seoul Bike-Sharing, World Cities, Bike-Sharing Systems & Weather Forecasts	1
How to read this report	1
1. Executive Summary	1
2. Seoul Bike-Sharing Dataset	2
3. Global Bike-Sharing Systems Dataset	14
4. Cross-Dataset: World Cities × Bike-Sharing Systems	19
5. Weather Forecast Dataset	22
6. Three-Way Cross-Dataset Visualizations (Population × Fleet × Weather)	23
7. Consolidated Findings & Data-Quality Log	28
8. Recommendations for the Data Science Phase	30

Statistical Analysis Report

Notebook-Based EDA: Seoul Bike-Sharing, World Cities, Bike-Sharing Systems & Weather Forecasts

Source: statistical_analysis.ipynb — R kernel (tidyverse, dplyr, ggplot2, scales, stringr), 95 cells (37 code, 58 markdown) **Report date:** September 18, 2026

How to read this report

This report walks through a visual, R/ggplot2-based exploratory analysis notebook covering the same four underlying tables examined in the earlier SQL-based report (`processed_seoul_bike_sharing.csv`, `processed_bike_sharing_systems.csv`, `processed_worldcities.csv`, `processed_cities_weather_forecast.csv`). Where the SQL report worked from raw aggregate query output, this one works from the notebook author's own code, printed statistics, and 31 generated charts — reproduced below with the original captions/narrative alongside independent commentary.

As before:

- Every **NULL/NA** encountered is flagged and **deferred to the next DATA SCIENCE phase** rather than imputed or guessed at here.
 - Where this notebook's own joins/visualizations show the same **data-quality issues identified in the SQL report** (city-name fan-out, missing values, etc.), that's called out explicitly, with a few *new*, chart-specific issues this notebook surfaces that the SQL dump did not (most notably, a cross-dataset chart that silently drops Seoul — see Section 6).
-

1. Executive Summary

- The notebook's primary dataset (`SEOUL_BIKE_SHARING`, loaded as `dataset`) is confirmed **fully clean at the row level** — `sum(is.na(dataset))` **returns 0** — but is only “clean” because non-functioning days were already filtered out upstream: `FUNCTIONING_DAY == "Yes"` matches all 8,465 rows.

- Of the 8,760 hours a complete Dec 2017–Nov 2018 year should contain, **295 are missing (288 from 12 fully-missing calendar dates, plus 7 single missing hours scattered across otherwise-complete dates)** — precisely recovered from the `summary()` output’s per-hour factor counts.
- Demand is heavily **right-skewed** (median 542 vs. mean 729.2 rentals/hour) and strongly **seasonal and diurnal** — every chart in the notebook that slices by hour or season reproduces the same two-peak (08:00 / 18:00) commuter pattern found in the SQL report.
- Weather sensitivity plots show the same **“Goldilocks zone”** pattern independently confirmed here: rentals rise with temperature up to a peak around 20–30 °C, then taper; a similar rise-then-fall pattern appears for humidity (~50%) and wind speed (~3 units).
- The **largest bike-sharing fleets** chart surfaces a fresh data-quality catch not visible in the SQL dump: one “city” entry is actually the string **“Town and cities in the Netherlands: all over the country, mainly at railway stations”** — a national description masquerading as a city name — and five different cities (Zhuzhou, Xi’an, Weifang, Seoul, Kunshan) share the suspiciously round fleet value of **exactly 20,000**.
- The three-way **population × bike-fleet × weather** visualizations (the notebook’s own “main three-dataset visualization”) reduce, after the joins, to just **three data points** — and **Seoul itself does not appear among them**, despite being present in all three source tables individually. This is a new, chart-specific instance of the join fan-out problem documented in the SQL report, and it undercuts the notebook’s flagship cross-dataset chart. See Section 6.3.
- A trend line in one chart (bicycles-per-1,000-people vs. forecast temperature) is very likely a **statistical artifact of pseudo-replication**: it’s fit across repeated 3-hourly readings from only two or three cities, not independent city-level observations, and should not be read as a genuine temperature effect on infrastructure. See Section 6.4.

2. Seoul Bike-Sharing Dataset

2.1 Data Preparation & Completeness

The notebook parses DATE from "%d/%m/%Y" format and coerces HOUR into an ordered factor, then checks:

```
sum(is.na(dataset)) # -> 0
summary(dataset)
```

Zero missing values in the processed table. The `summary()` output gives a clean statistical snapshot:

Variable	Min	1st Qu.	Median	Mean	3rd Qu.	Max
RENTED_BIKE_COUNT		214	542	729.2	1,084	3,556
TEMPERATURE (°C)	-17.80	3.00	13.50	12.77	22.70	39.40
HUMIDITY (%)	0	42	57	58.15	74	98
WIND_SPEED	0	0.90	1.50	1.726	2.30	7.40
VISIBILITY	27	935	1,690	1,434	2,000	2,000
DEW_POINT_TEMPERATURE (°C)	-5.10		4.70	3.945	15.20	27.20
SOLAR_RADIATION	0	0	0.01	0.568	0.93	3.52
RAINFALL (mm)	0	0	0	0.149	0	35.00

Variable	Min	1st Qu.	Median	Mean	3rd Qu.	Max
SNOWFALL (cm)	0	0	0	0.078	0	8.80

A few things worth calling out beyond what the notebook itself notes:

- **RENTED_BIKE_COUNT's median (542) sits well below its mean (729.2)** — the right-skew visible later in the histogram (Section 2.3) is already evident here.
- **VISIBILITY is right-censored at 2,000** (median 1,690, max exactly 2,000, and 2,000 is also the 3rd quartile) — this looks like a sensor/reporting ceiling rather than a true maximum, worth flagging for the next phase rather than treating 2,000 as a real physical bound.
- **RAINFALL and SNOWFALL both have a median and 1st/3rd quartile of exactly 0** — confirming the notebook's own observation that precipitation "only happens in the fourth quartile."
- The processed file carries **44 columns**, including pre-computed one-hot-encoded versions of SEASONS, HOLIDAY, and HOUR (e.g. SEASONS_SUMMER, HOLIDAY_HOLIDAY, HOUR_0...HOUR_23) alongside their original categorical forms — this table has clearly been pre-staged for downstream modeling, not just EDA.

Completeness check: the notebook computes $\text{hours} \times 365 = 8,760$ as the expected record count, versus **8,465 actual rows where FUNCTIONING_DAY == "Yes"** — a shortfall of 295 hours. Cross-referencing the `summary()` table's HOUR factor counts (hours 7-12 each show exactly 353 records, while the remaining 18 "Other" hours sum to 6,347) makes it possible to pin the shortfall down precisely: **288 hours come from 12 fully-missing calendar dates** (365 possible dates – 353 present = 12), and the remaining **7 hours are single gaps scattered across 7 different hours-of-day** on otherwise-complete dates ($18 \times 353 = 6,354$ expected vs. 6,347 actual). **This 295-hour gap is a completeness issue, not a NULL value, and is noted here rather than resolved.**

Also notable: since `nrow(filter(dataset, FUNCTIONING_DAY == "Yes"))` equals the full row count (8,465), **every row in this processed file is already a "functioning day."** This explains why the dataset has no zero-rental hours (minimum is 2) — any non-functioning-day rows (which would show 0 rentals) were filtered out before this file was produced.

2.2 Holidays & Seasonal Precipitation

```
filtered_df <- dataset %>% filter(HOLIDAY == "Holiday")
holiday_count <- count(filtered_df) # 408
holiday_perc <- (holiday_count/nrow(dataset)) * 100 # 4.82%
```

408 holiday hours (4.82% of the dataset) — matching the SQL-based report's independently-derived figure exactly, which is a good cross-validation between the two analyses.

Seasonal precipitation totals (`sum(RAINFALL)`, `sum(SNOWFALL)` grouped by SEASONS):

Season	Total rainfall (mm)	Total snowfall (cm)
Autumn	227.9	123.0
Spring	403.8	0.0
Summer	559.7	0.0
Winter	70.9	534.6

The notebook’s own reaction — “*That seems like a lot of snow*” — is a reasonable one: **Winter alone accounts for 534.6 of the ~658 cm total snowfall in the year (over 80%)**, as expected, but **Autumn’s 123.0 cm of snow is the more surprising figure**, since autumn is not typically a snow season; this is consistent with an early/late-season snow event right at the Nov 30 boundary of the dataset (visible directly in the chart in Section 2.7).

2.3 Distribution of the Target Variable

“Taking a closer look at our main variable of interest, namely, `RENTED_BIKE_COUNT`... it is a measured quantity, and we expect it to depend on factors such as the expected weather.”

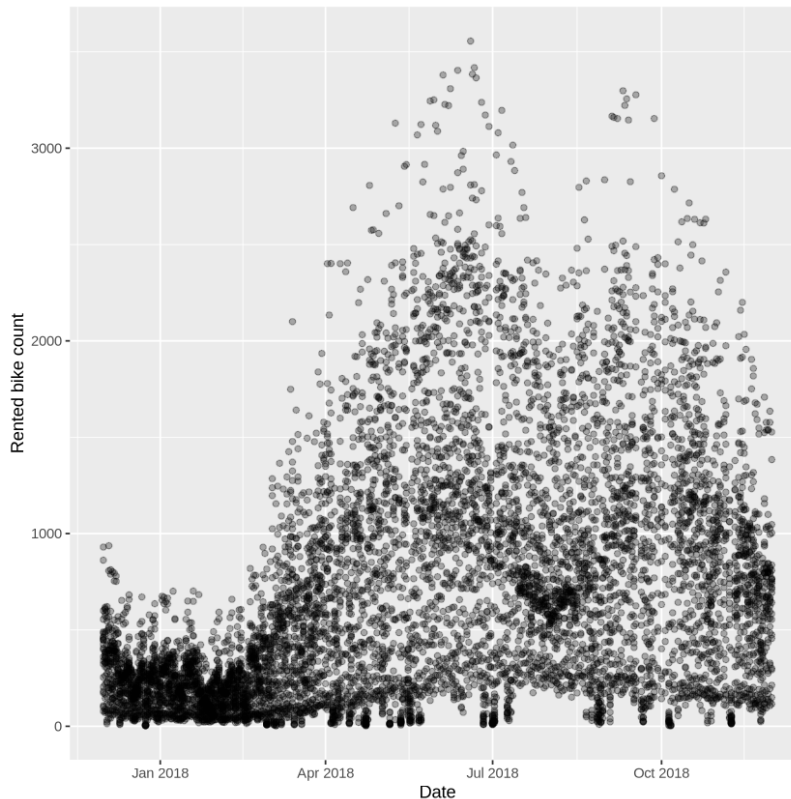


Figure 1: Rented bike count over time

The full year of hourly observations, plotted as-is. The seasonal envelope is immediately visible: a low, flat band through the winter months, rising sharply from March, peaking through summer, and declining again into autumn — visually confirming the $\sim 4.6\times$ summer/winter demand ratio quantified in the SQL report.

“Bike demand depends strongly on both season and hour of day with the majority being in the summer, in the afternoon.” Adding `HOUR` as a color channel shows the **highest points of the year are dominated by green/yellow-green tones (roughly hours 17-21)** — a visual confirmation that the evening commute window drives the most extreme demand spikes, not just typical demand.

“We can see from the histogram that most of the time there are relatively few bikes rented. Indeed, the ‘mode’... is about 250. Judging by the ‘bumps’ at about 700, 900, 1900, and 3200 bikes, it looks like there may be other modes hiding within subgroups of the data.” This is an astute read of the shape: a classic right-skewed distribution with a sharp peak near 200-300 and a long tail, plus subtle secondary bumps consistent with the distribution being a



Figure 2: Rented bike count over time, colored by hour

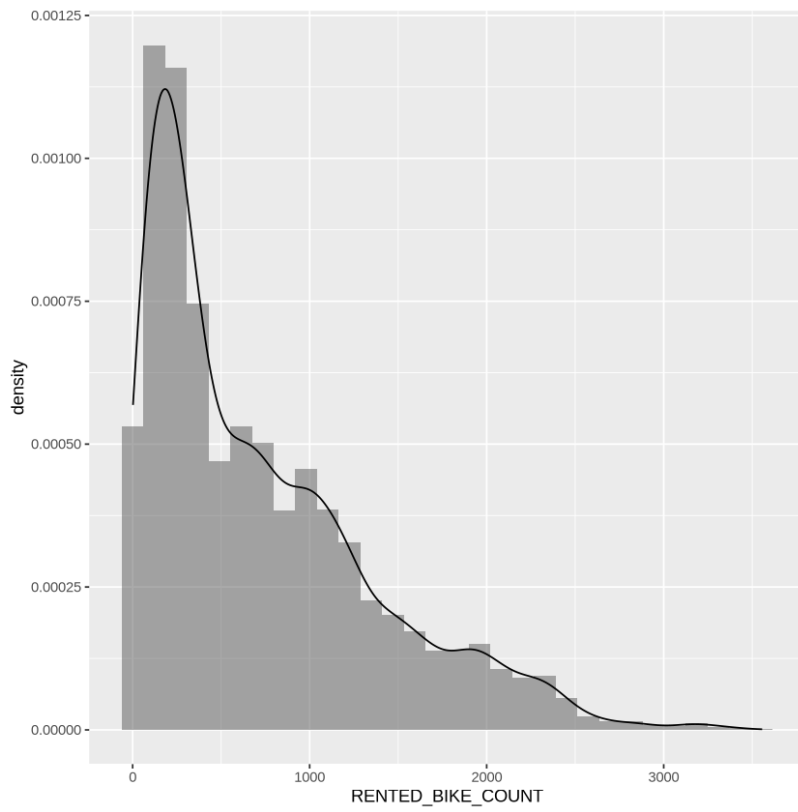


Figure 3: Distribution of rented bike count (density histogram)

mixture of several very different regimes (winter nights vs. summer rush hours) rather than one smooth population.

2.4 Rentals by Temperature, Season & Hour

“I start with `RENTED_BIKE_COUNT` vs. `TEMPERATURE`, then I generate four plots corresponding to the `SEASONS`... I used `HOUR` as the color.”

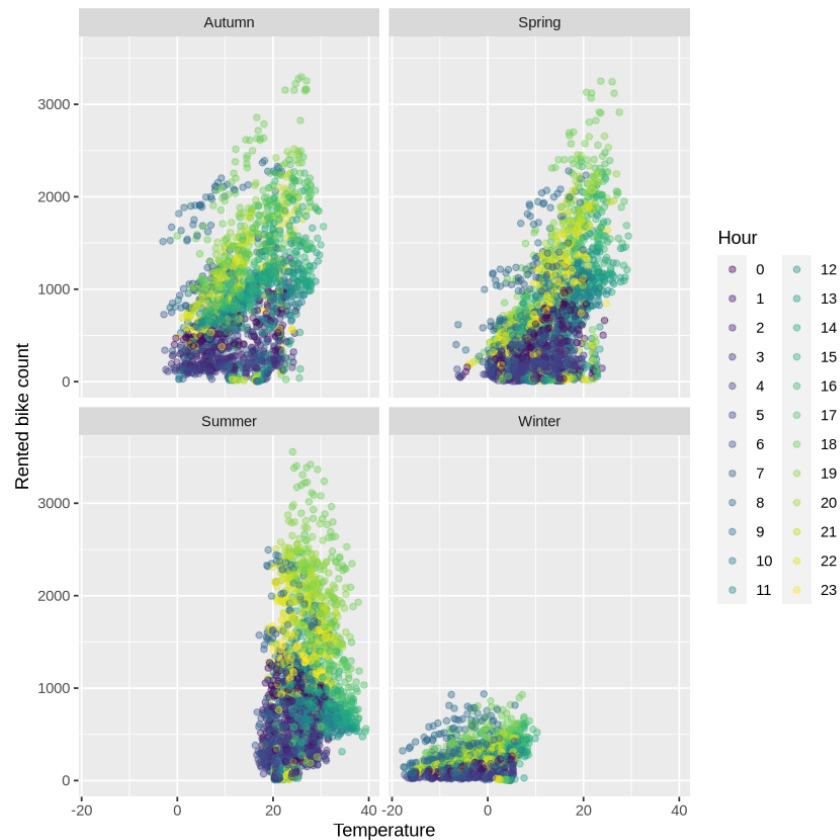


Figure 4: Rentals vs temperature, faceted by season, colored by hour

“Visually, we can see some strong correlations as approximately linear patterns.” Autumn and Spring both show a clean rising trend with temperature; Summer compresses into a narrow high-temperature band; Winter compresses into a narrow low-temperature, low-count band. In every panel, the highest-count points skew toward the green/yellow (evening) hour colors.

“Comparing this plot to the same plot... without grouping by `SEASONS`, shows how important seasonality is in explaining bike rental counts.” Collapsing the four season panels into one scatter blurs the clean linear trends seen above into a single fan-shaped cloud — a good illustration of Simpson’s-paradox-adjacent behavior: season is a real confounder for the temperature relationship.

“Although the overall scale of bike rental counts changes with the seasons, key features remain very similar. For example, peak demand times are the same across all seasons, at 8 am and 6 pm.” This is the clearest single chart in the notebook for the two-peak commuter pattern — every season’s boxplot median rises through the morning, dips slightly at midday, and peaks again around 17:00–19:00, with the evening peak visibly higher than the morning one in Autumn, Spring, and Summer (Winter is comparatively flat throughout, consistent with its much lower overall demand).

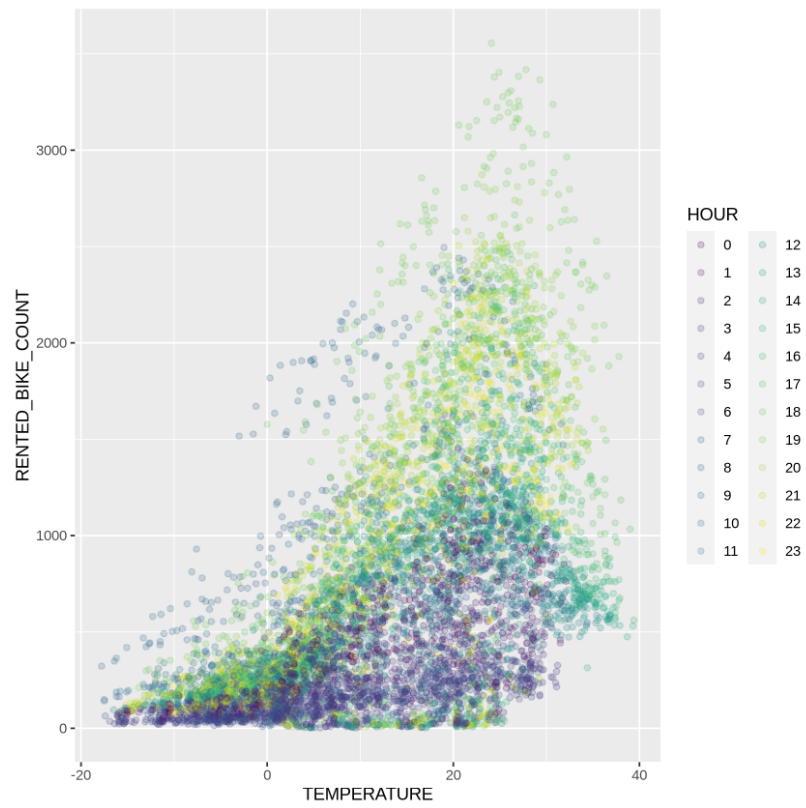


Figure 5: Rentals vs temperature, unfaceted

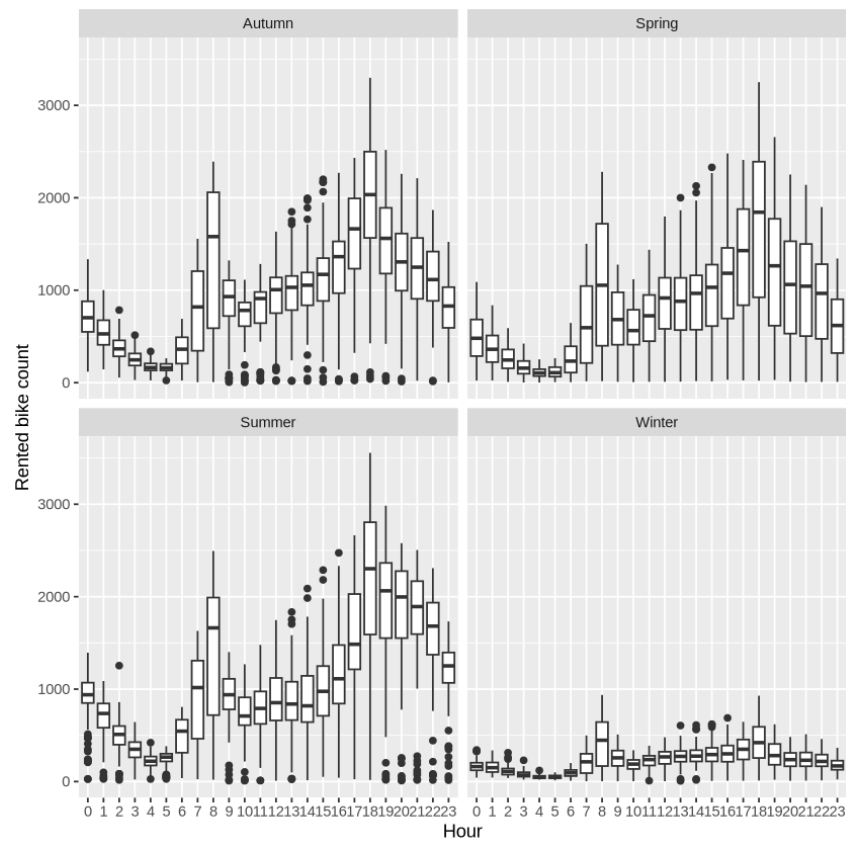


Figure 6: Boxplots of rentals by hour, faceted by season

2.5 Rentals by Season & Holiday Status

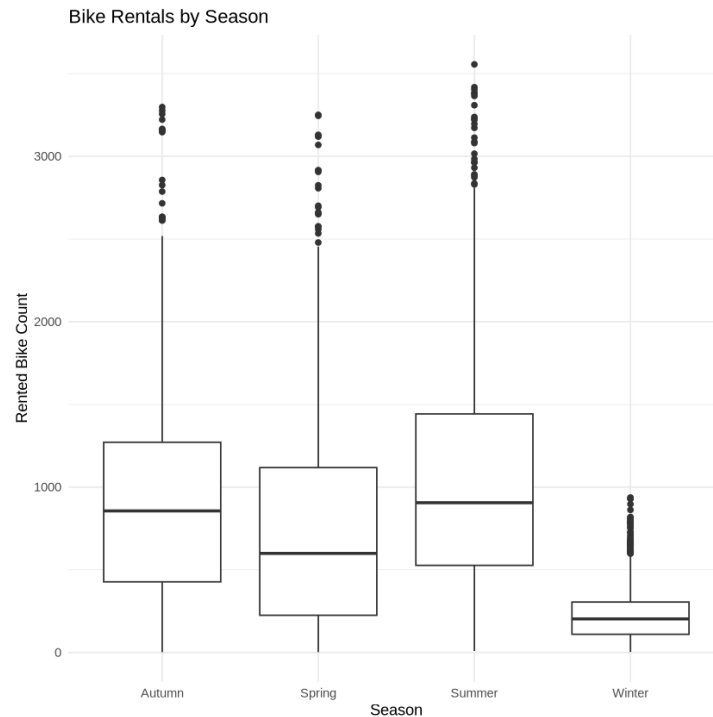


Figure 7: Boxplot of rentals by season

“Lots of outliers and positioning of IQR indicates heavy skew to the right.” Summer has both the highest median and the most extreme outliers (up to 3,556); Winter’s box is visibly compressed near the bottom of the axis.

“No holiday distribution has more spread compared to holiday and also more severe outliers. Both distributions indicate severe skewness to the right.” Regular (“No Holiday”) days show a materially higher median and a much longer whisker/outlier tail (up to 3,556) than holidays (topping out near 2,400) — consistent with the ~28% average demand reduction on holidays quantified in the SQL report.

2.6 Weather Sensitivity

“A scatter plot can show whether temperature and bike rental activity appear to be related.”

“This indicates a positive correlation between temperature and rented bike count... Interestingly the trend also tends to reverse with extreme temperatures like close to 40 degrees... indicating a goldylocks zone between 20 and 30 degrees.” The fitted LOESS curve rises steadily from near 0 at -18 °C to a peak around 1,100-1,150 near 25 °C, then very gently declines toward 40 °C — closely matching the SQL report’s bucketed finding that 20-30 °C is the single best-performing temperature band.

“Similar pattern for humidity levels. With the goldylocks zone being at around 50 humidity.” The curve peaks gently around 40-60% humidity before declining sharply above ~80% — again matching the SQL report’s finding that the 90%+ humidity bucket sees a ~77% demand drop relative to the peak band.

“Similar pattern as before with the goldylocks zone being at around 3 wind speed.” Worth flagging: wind speed itself likely correlates with other weather variables (e.g., stormier/rainier

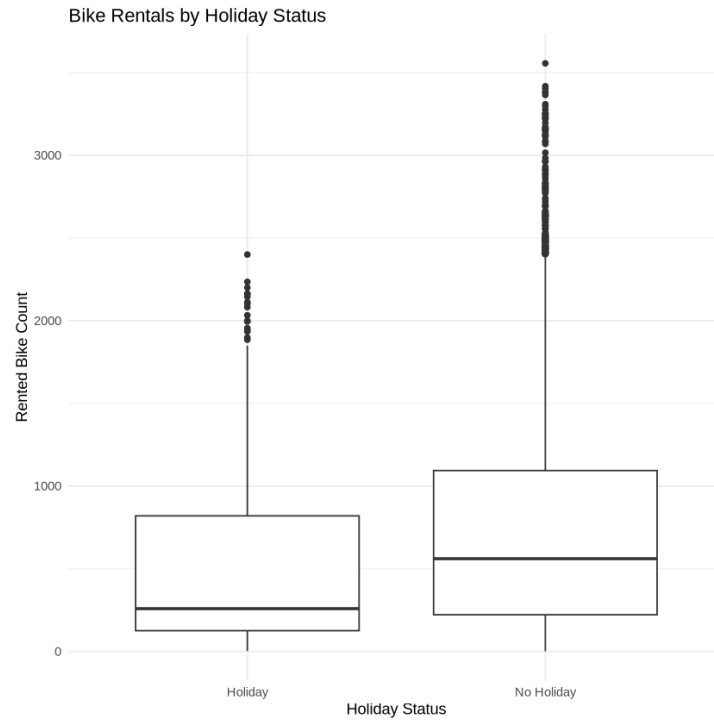


Figure 8: Boxplot of rentals by holiday status

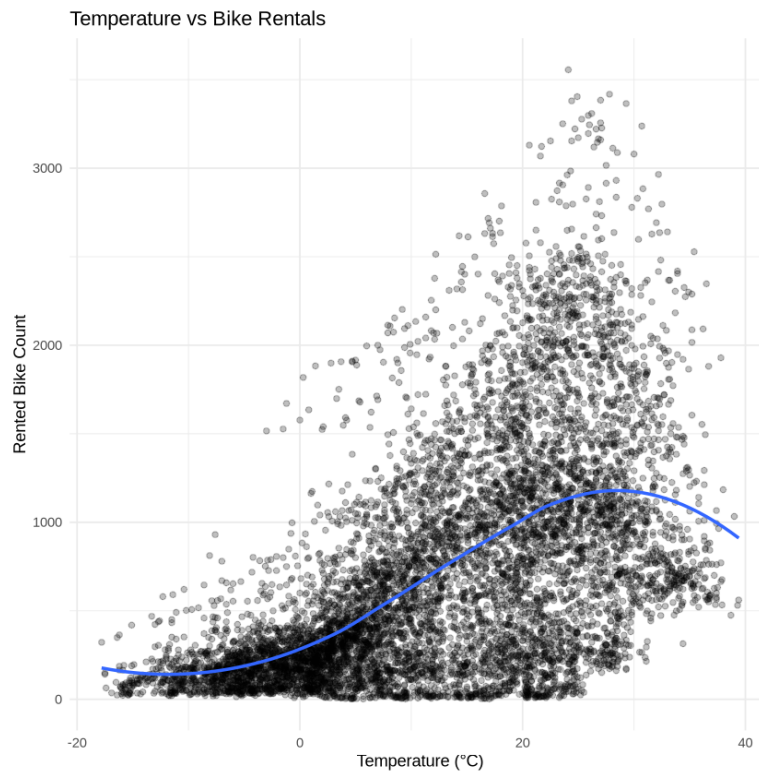


Figure 9: Temperature vs rentals with LOESS smooth

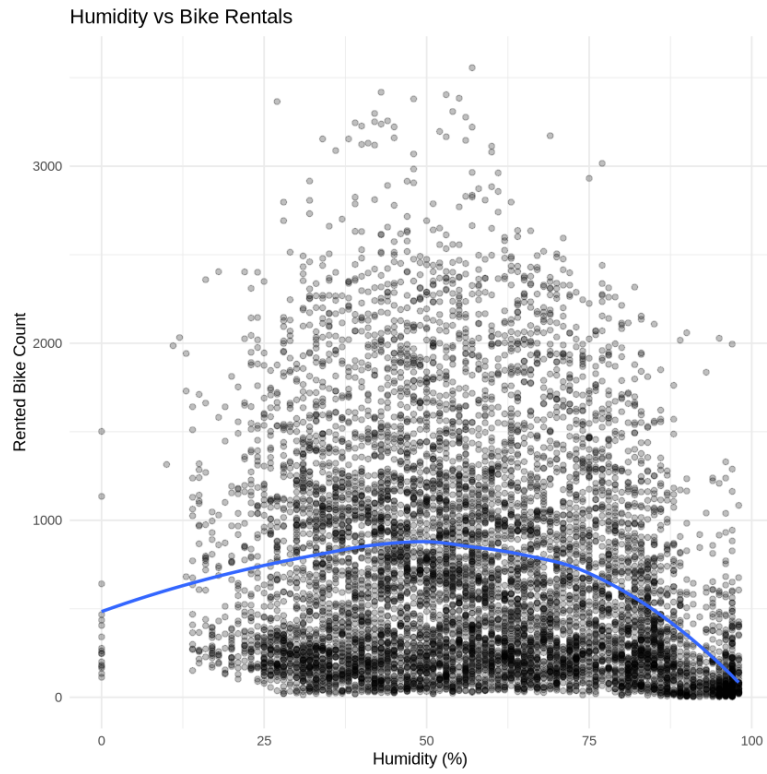


Figure 10: Humidity vs rentals with LOESS smooth

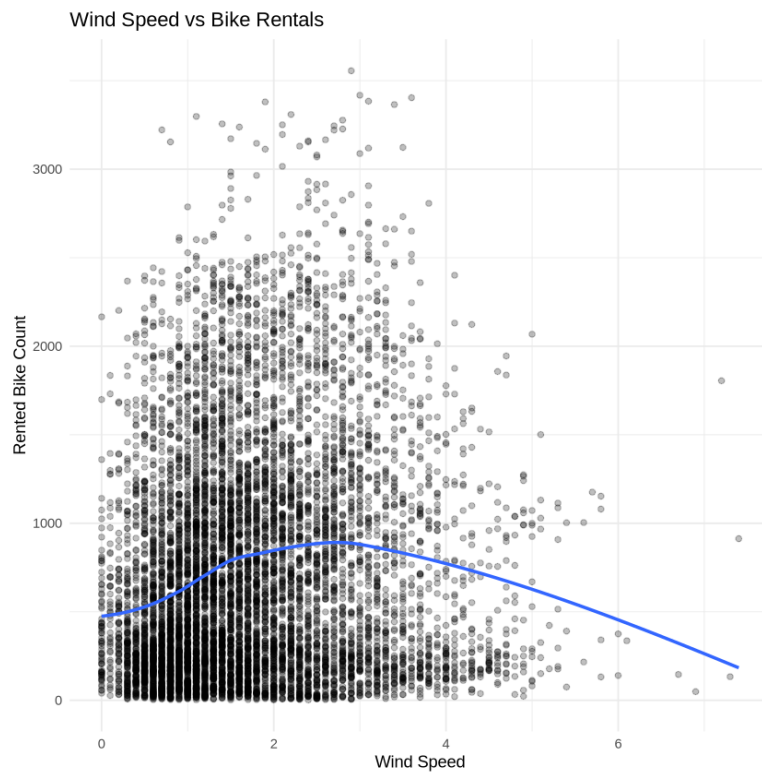


Figure 11: Wind speed vs rentals with LOESS smooth

conditions), so this curve may partly reflect a **confounded relationship rather than a direct causal effect of wind on ridership** — the notebook doesn't control for this, and it's a reasonable next step for the Data Science phase.

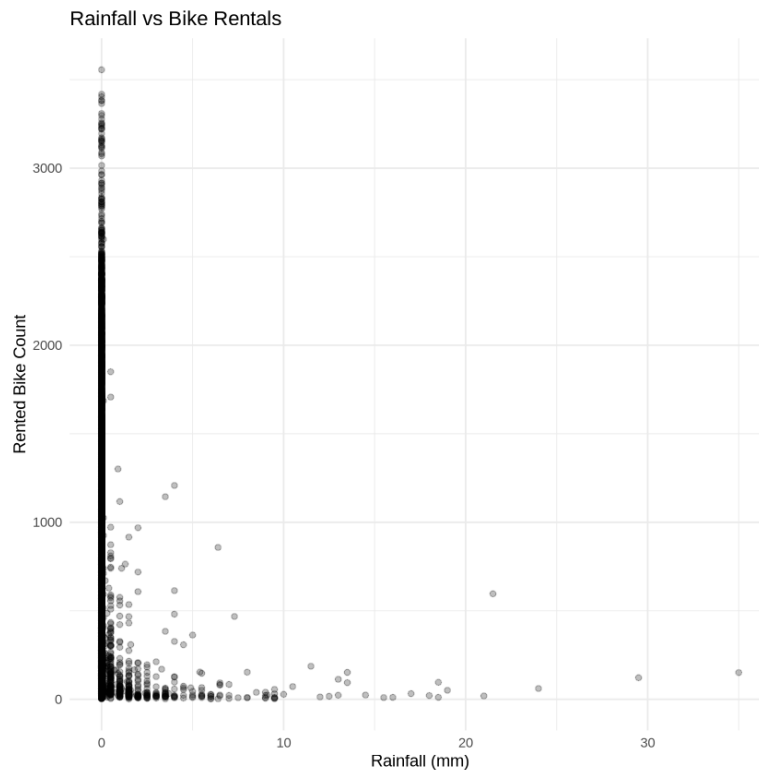


Figure 12: Rainfall vs rentals

“Pretty much the majority of bike rides occur with no rain, with a few outliers happening during rain. This indicated the distribution is severely skewed to the right.” Note: the LOESS smoother **failed to compute here** (Caused by error in `predLoess()`: NA/NaN/Inf in foreign function call) — the notebook's underlying warning log shows this explicitly, and no trend line appears in the chart as a result, purely because rainfall values are so overwhelmingly concentrated at 0 that the local regression has too little spread to work with. This is a technical/plotting limitation, not a finding about the data itself.

“Similar pattern to rainfall. But notably there are more outliers with this weather condition. It looks like people prefer to ride bikes in snow rather than rain.” The same LOESS failure occurs here for the same reason. The visual comparison the notebook draws (more non-trivial rental activity persisting at low snowfall vs. low rainfall) is plausible directly from the point cloud, though it's a qualitative read rather than a formally tested claim.

2.7 Daily Precipitation Over Time

“I group the data by DATE... to calculate the daily total rainfall and snowfall... and plot the results using a line graph to track amounts over time.”

“Weather events are relatively infrequent but can produce substantial daily rainfall or snowfall, making precipitation a potentially important factor to consider when explaining unusually low bike-rental days.” This chart also makes the **Section 2.2 Autumn-snowfall anomaly visible directly**: there's a sharp, isolated snowfall spike right at the end of the plotted year (around the Nov 2018 boundary) reaching nearly 80 cm/day-equivalent — almost certainly the

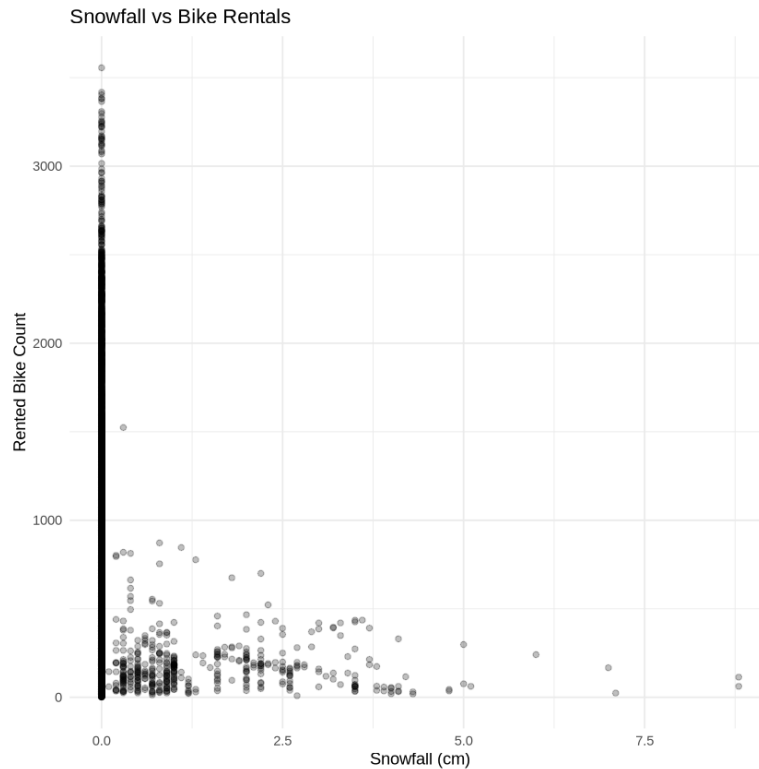


Figure 13: Snowfall vs rentals

source of Autumn’s unexpectedly high 123.0 cm snowfall total, and a single-day event worth separating out from “typical” autumn conditions if this data is used for seasonal modeling.

2.8 Hourly Averages

“This shows how the number of rented bikes changes throughout the year. Instead of plotting every observation, this summarizes the typical rental activity for each hour.”

“This shows spike of activity in the morning and in the afternoon. Also shows dips of inactivity late in the evening and around midnight.” (Note: `geom_line()` produced a warning here — “Each group consists of only one observation” — because `HOUR` is a factor with no continuous grouping variable, so the chart renders as points only, without the connecting line the code requested. Cosmetic, not a data issue.) The point pattern matches the SQL report’s hour-by-hour averages almost exactly: a trough near 04:00–05:00, a morning shoulder around 08:00, and the year’s single highest point at 18:00.

3. Global Bike-Sharing Systems Dataset

3.1 Largest Individual Fleets

“This gives me a top-15 city comparison rather than trying to display hundreds of cities at once.”

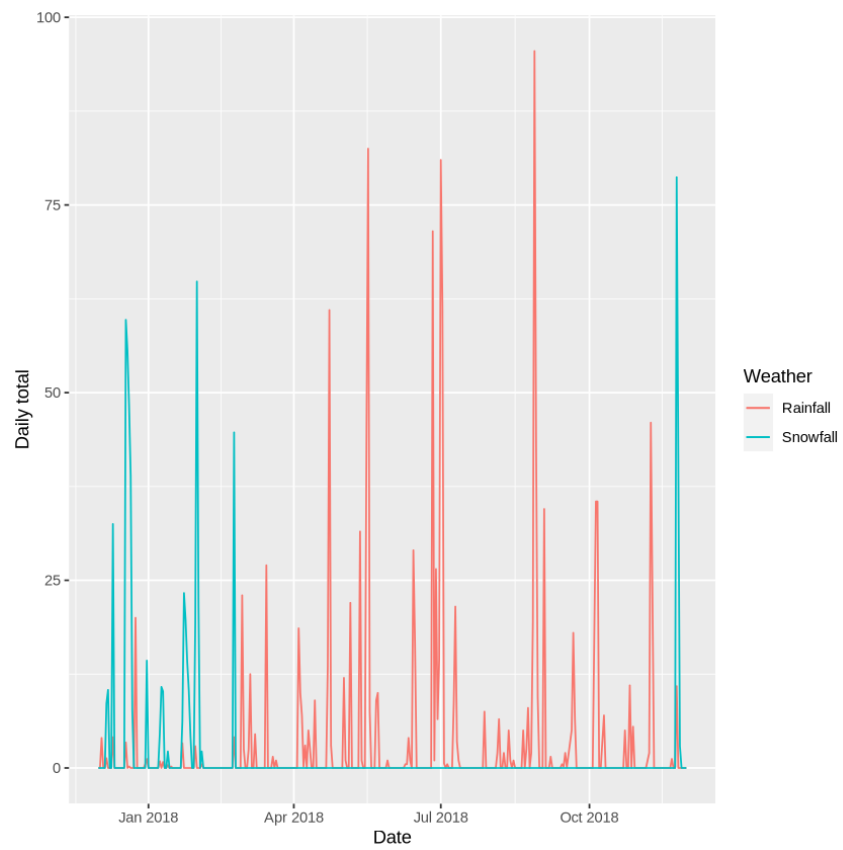


Figure 14: Daily total rainfall and snowfall over the year

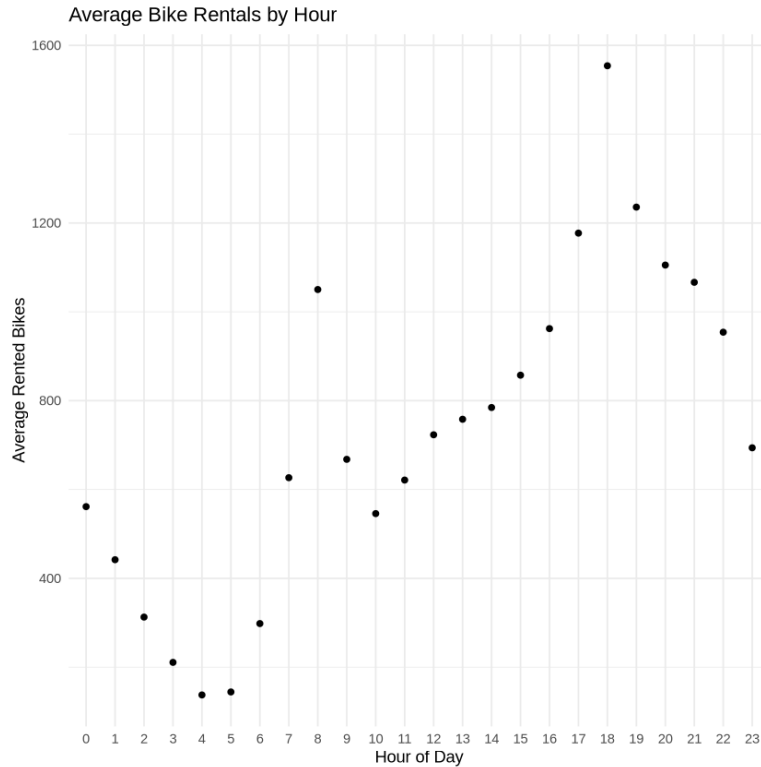


Figure 15: Average bike rentals by hour

Rank	City	Bicycles
1	Hangzhou	78,000
2	Taiyuan	41,000
3	Suzhou	22,940
4	<i>"Towns and cities in the Netherlands: all over the country, mainly at railway stations"</i>	21,700
5	Zhuzhou	20,000
5	Xi'an	20,000
5	Weifang	20,000
5	Seoul	20,000
5	Kunshan	20,000
10	Shanghai	19,165
11	Xuzhou	18,000
12	Beijing	16,000
13	Ningbo	15,000
14	Paris	14,500
15	London (England)	13,600

"It appears bike riding is enjoyed more in ASIA." Fair read — 8 of the top 13 entries are Chinese cities. Two data-quality points worth flagging that the notebook's own narrative doesn't mention:

- **Rank 4 is not a city at all.** *"Towns and cities in the Netherlands: all over the country,*

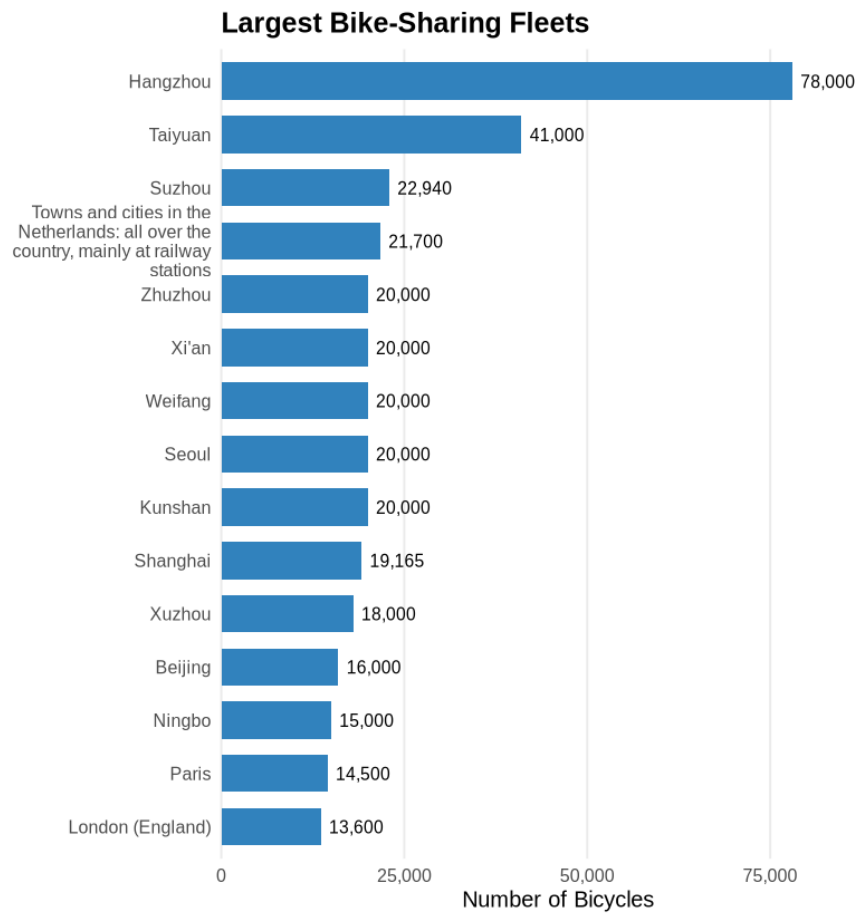


Figure 16: Largest bike-sharing fleets by city

mainly at railway stations” is a national-level description that has ended up in the CITY field, being treated as if it were a single city with 21,700 bicycles. **To be addressed in the next DATA SCIENCE phase** — this record should be re-classified or excluded from any city-level ranking.

- **Five different cities (Zhuzhou, Xi’an, Weifang, Seoul, Kunshan) all report the exact same round fleet size of 20,000.** Round-number ties across otherwise-unrelated cities are a common sign of placeholder or default values rather than five independently-measured fleet counts — worth verifying against source data rather than assuming precision.

3.2 Fleet by Country

“I also aggregate the bike-sharing systems by country.”

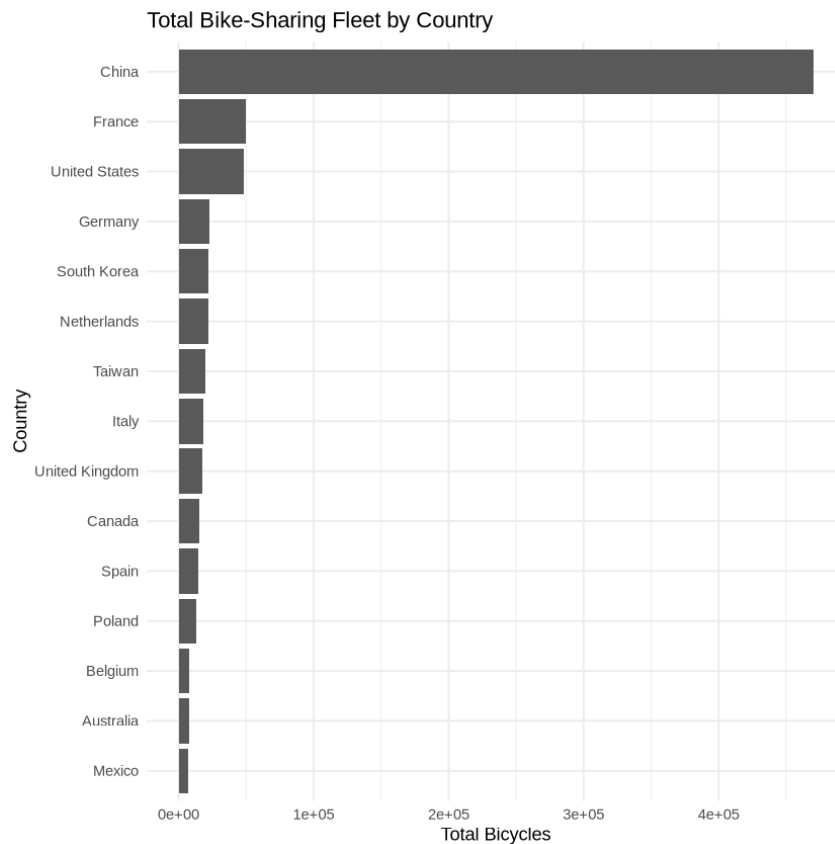


Figure 17: Total bike-sharing fleet by country (top 15)

“Overwhelming advantage in bike fleet numbers for China.” This matches the SQL report’s country-level totals precisely: **China (469,941), France (49,552), and the United States (48,008)** lead, with China’s total alone representing roughly 58% of the combined fleet across all countries with known totals.

4. Cross-Dataset: World Cities × Bike-Sharing Systems

4.1 Population vs. Fleet Size

“Do cities with larger populations also tend to have larger bike-sharing fleets?”



Figure 18: Population vs bike-sharing fleet size, log-log with linear fit

```
population_fleet <- dataset1 %>%  
  inner_join(dataset2 %>% select(CITY, COUNTRY, POPULATION), by = c("CITY", "COUNTRY")) %>%  
  filter(!is.na(BICYCLES), !is.na(POPULATION), POPULATION > 0)
```

“The log scales are useful here because both population and fleet size have very large ranges. The plot also indicates a positive correlation between population and bike fleet size.” The positive log-log trend is real and visible. **Important caveat the notebook’s narrative doesn’t surface:** even though this join uses the safer CITY + COUNTRY key (matching the SQL report’s own recommended fix), R still raises a many-to-many join warning here:

“Detected an unexpected many-to-many relationship between x and y. Row 39 of x matches multiple rows in y. Row 142 of y matches multiple rows in x.”

This confirms something the SQL report could only partially establish: **the fan-out problem isn’t solely caused by identical city names across different countries (like the many “Paris”es) — WORLD_CITIES also contains duplicate CITY+COUNTRY rows for the same real place** (e.g., the two Suzhou, China entries identified in the SQL report’s Section 6.2), which is enough on its own to trigger a many-to-many join even when COUNTRY is included as a join key. **The underlying duplicate-row problem in WORLD_CITIES needs to be fixed at the source, not just patched by adding COUNTRY to the join.**

4.2 Bicycles per 1,000 People

“This normalizes fleet size by population... different from simply looking at the largest fleets because it accounts for population size.”

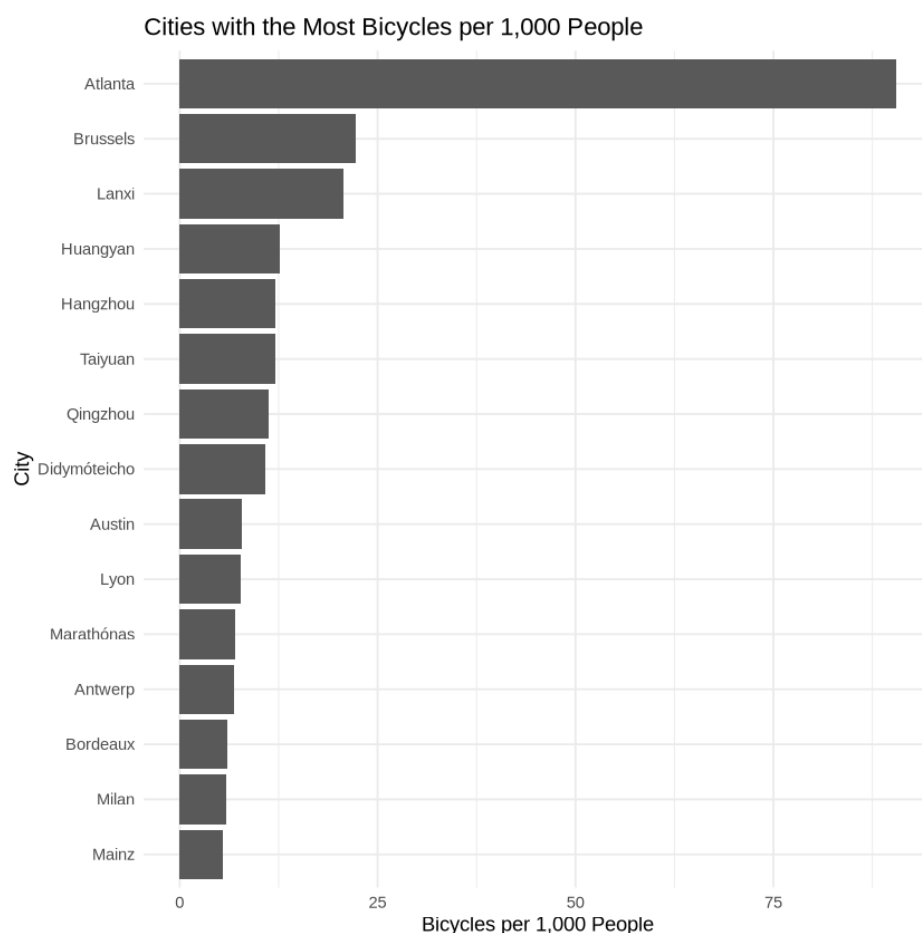


Figure 19: Cities with the most bicycles per 1,000 people (top 15)

Atlanta tops this list at roughly 90 bicycles per 1,000 people — nearly 4× the second-place city (Brussels). This is the exact same artifact identified independently in the SQL report: Atlanta’s real fleet (500 bikes) is being divided by the population of a small, unrelated place also named “Atlanta” rather than metro Atlanta’s actual ~5.4 million residents (which would put the true figure closer to 0.09 per 1,000). **Lanxi**, in third place here, is the same city flagged in the SQL report for an identical reason. Both should be treated as **join artifacts, not genuine outliers**, pending the CITY+COUNTRY duplicate-row cleanup recommended above.

4.3 Population & Fleet by Country

“This visualization extends the population/fleet scatter plot from earlier.” The underlying log-log positive trend is still visible, though with ~50 countries in the legend the color encoding is difficult to read reliably — a chart-design note rather than a data issue: a top-N country subset or faceting would likely communicate this more clearly than a single 50-category color legend.

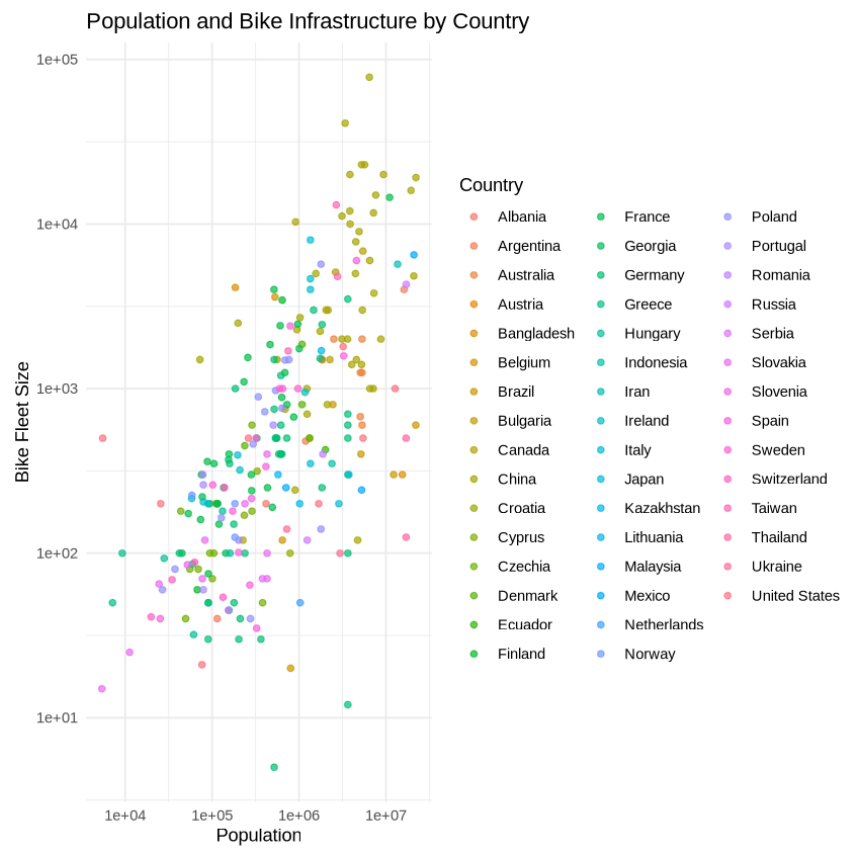


Figure 20: Population and bike infrastructure by country, colored

4.4 Geographic Distribution of Cities

“Since this dataset contains LAT and LNG, you can visualize the geographic distribution of the cities. A simple longitude/latitude scatter plot works without requiring a mapping package.”

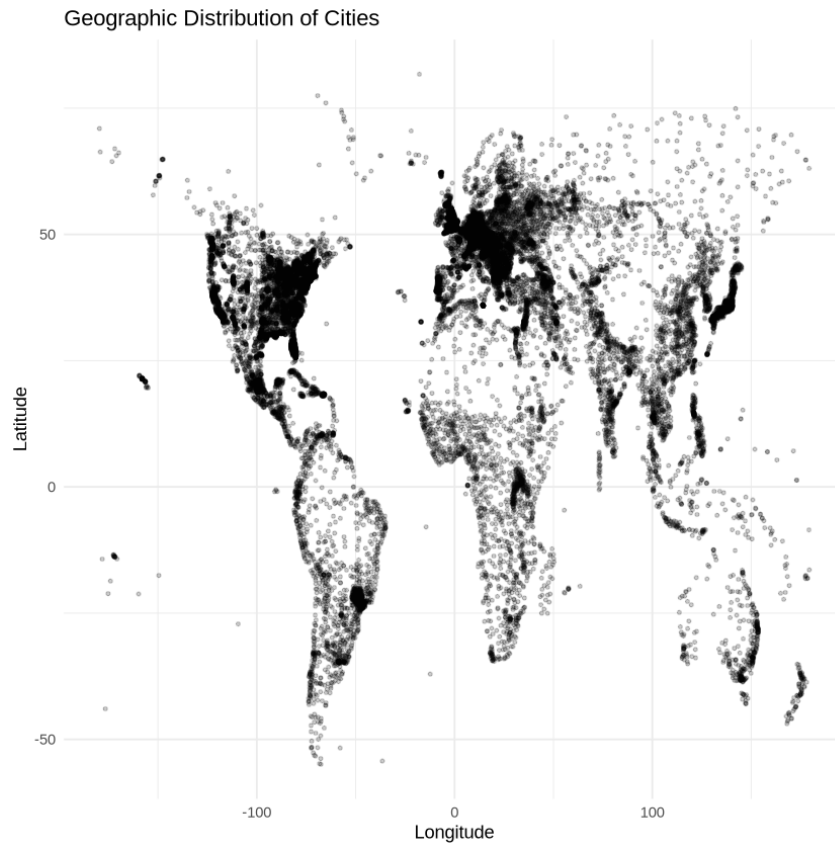


Figure 21: Geographic scatter of all world cities

A genuinely effective, package-free way to render a world map from point density alone — the continents are clearly recognizable. Worth noting in light of the SQL report’s Section 2.5: **the densest single cluster in the entire plot is the eastern/central United States**, visually corroborating the earlier finding that the U.S. contributes a disproportionate 29% of all city records (and 83% of duplicate CITY+COUNTRY rows) in this table.

5. Weather Forecast Dataset

5.1 Weather Condition Frequency

“The weather forecast dataset can be explored independently by looking at the different weather conditions.”

“It’s mostly cloudy or clear with very little rain.” Approximately 100 of the 160 forecast records show “Clouds,” ~52 show “Clear,” and only ~8 show “Rain” — consistent with the per-city breakdowns in the SQL report (Paris and Suzhou both skew heavily toward “Clouds”; Washington, D.C. skews toward “Clear”).

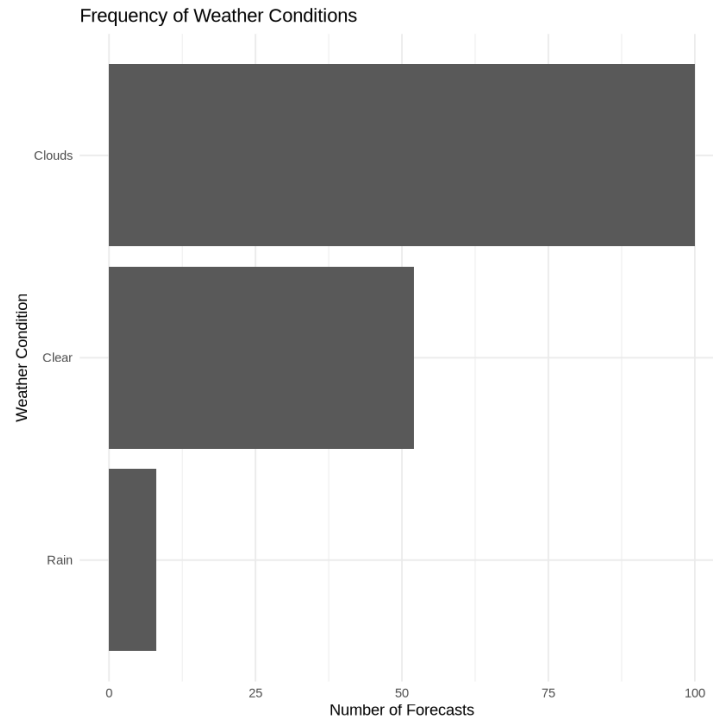


Figure 22: Frequency of weather conditions

5.2 Forecast Temperature Distribution

“Distribution looks close to a normal distribution but is slightly skewed to the right.” A reasonable read of the bulk of the distribution — but note the **single isolated bar separated from the rest of the histogram near 32-34 °C**. That’s almost certainly the same Paris 33.83 °C reading flagged in the SQL report as the dataset’s warmest single value, standing apart from the rest of the distribution by a visible gap rather than blending smoothly into the right tail.

5.3 Forecast Temperature by City

“Very few outliers. Some of the distributions are skewed to the left while others are skewed to the right.” **Paris is the only city with flagged outliers on this chart — three of them, all on the high side (roughly 29 °C, 32 °C, and 34 °C)**. This directly corroborates the SQL report’s finding of an unusually large (~24 °C) swing within Paris’s own forecast window, and visually isolates exactly which readings are driving that swing.

6. Three-Way Cross-Dataset Visualizations (Population × Fleet × Weather)

6.1 Fleet Size vs. Individual Weather Variables

“Bike fleet size vs forecast temperature. This visualization combines the bike infrastructure and weather forecast datasets.”

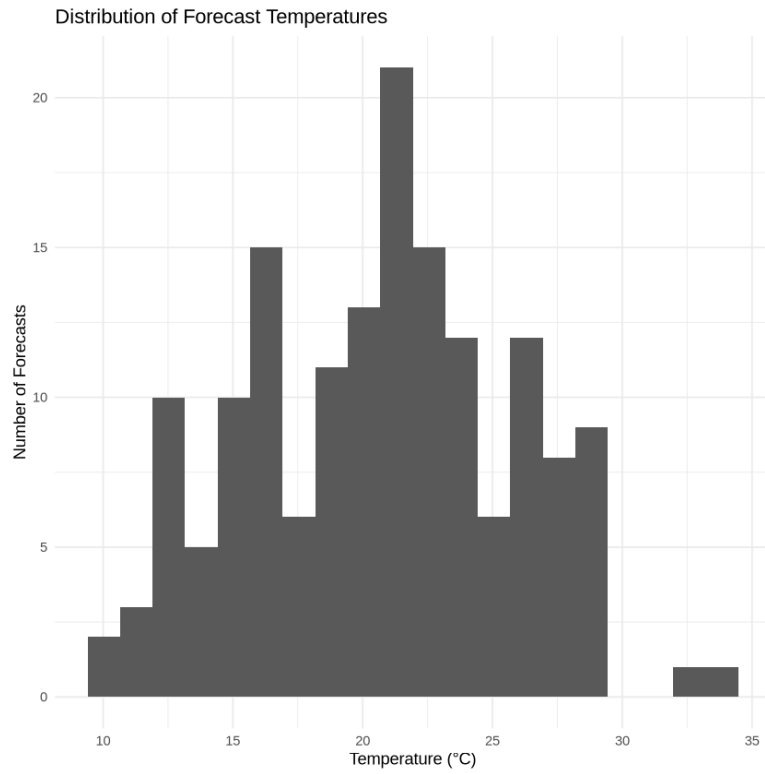


Figure 23: Distribution of forecast temperatures

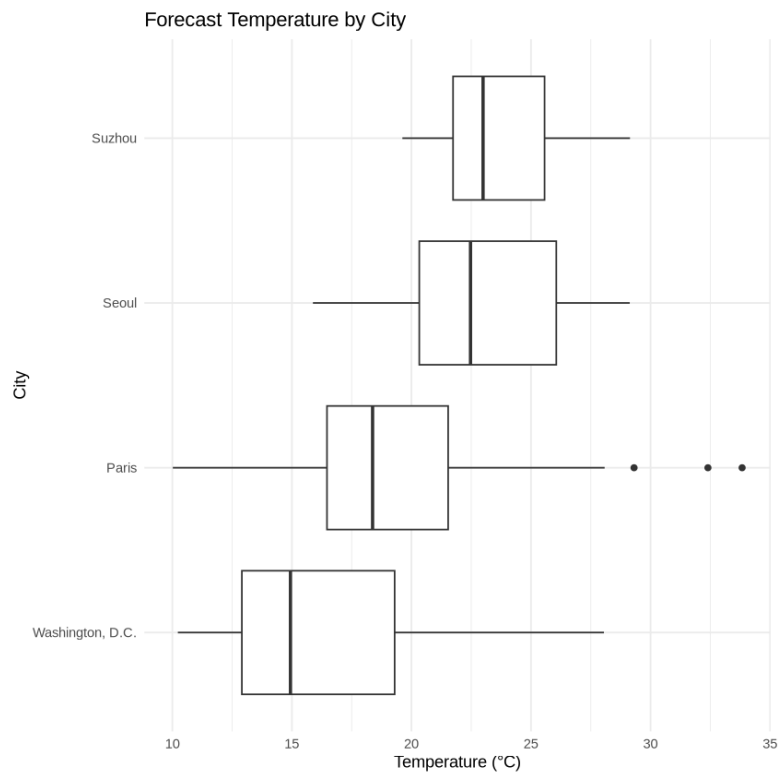
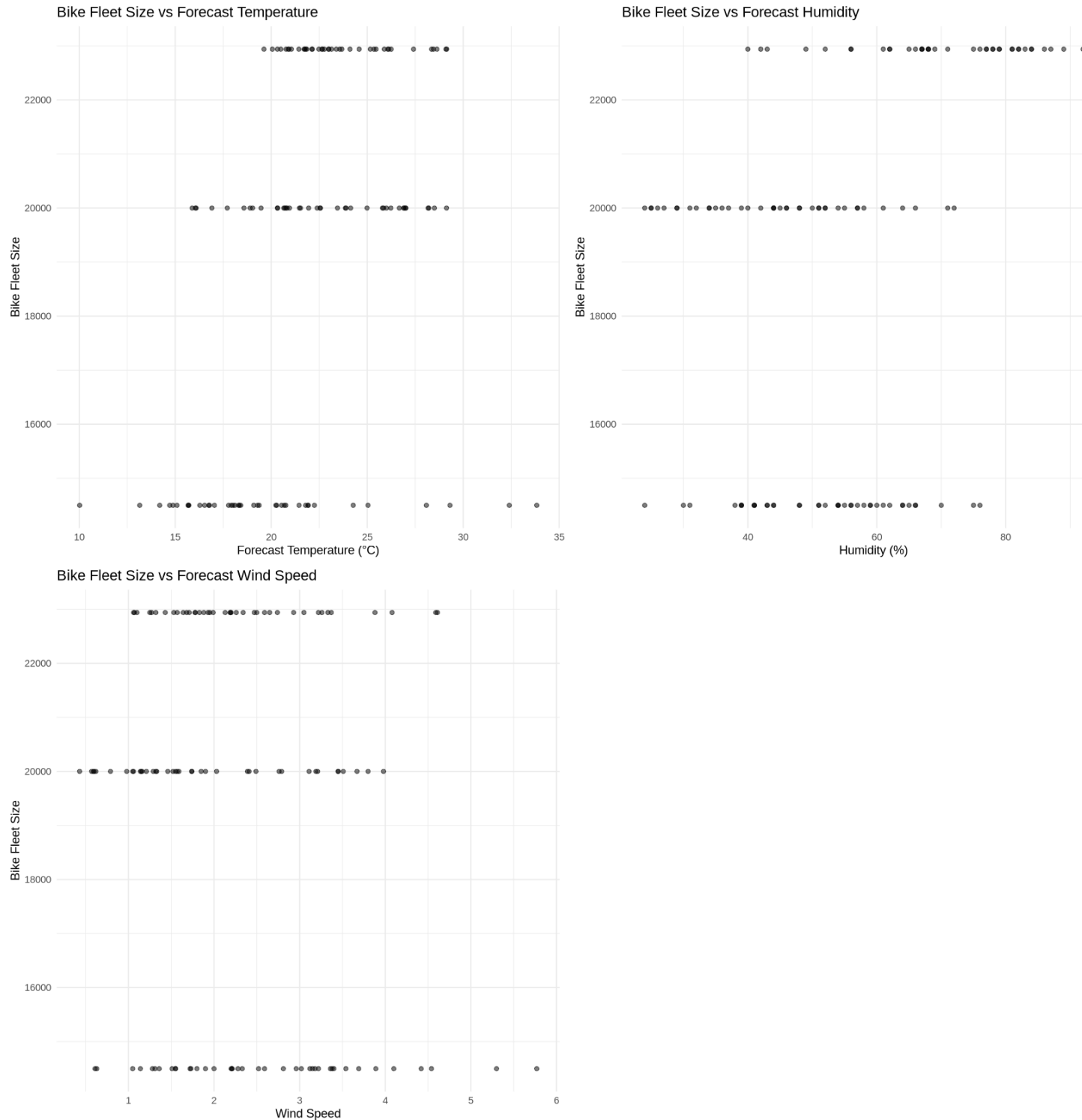


Figure 24: Forecast temperature by city, boxplot



These three charts join BIKE_SHARING_SYSTEMS to WEATHER_FORECAST by CITY alone (no COUNTRY needed, since there are only 3 matching cities). Because fleet size is **fixed per city** while the weather readings vary every 3 hours, each chart necessarily collapses into **three flat horizontal bands** — one per city (Paris $\approx 14,500$; Seoul = 20,000; Suzhou = 22,940) — spread out along the x-axis by that city's range of forecast readings. This is an accurate rendering of the data, but with only 3 distinct y-values, **these three charts can't reveal**

a real fleet-size/weather relationship — they mostly just show each city’s forecast temperature/humidity/wind-speed range. Useful as a sanity check, but not as evidence of any fleet-weather relationship.

6.2 Population, Fleet & Temperature — The “Main Three-Dataset Visualization”

“Population, bike fleet and weather together. This is the main three-dataset visualization.”

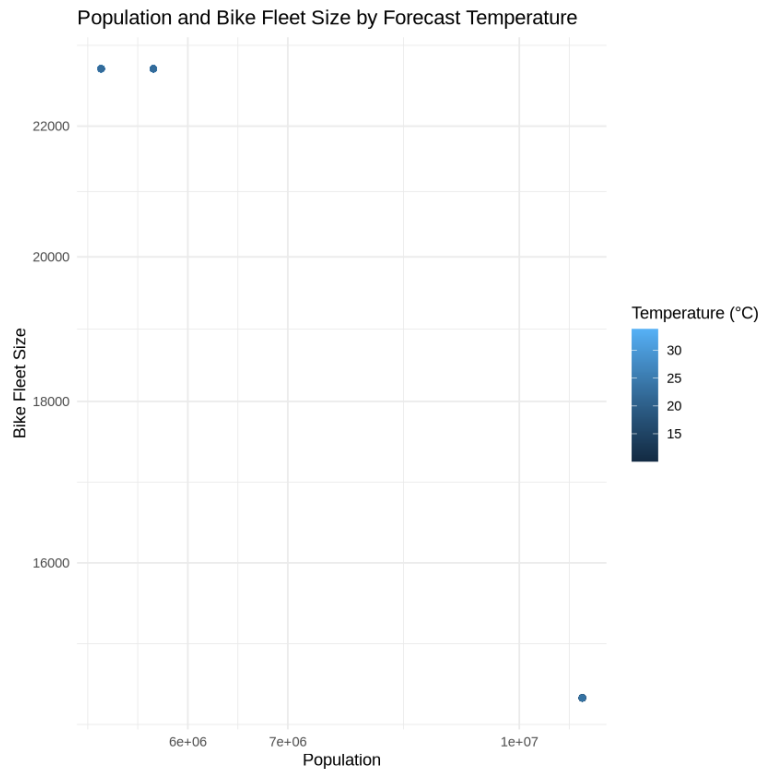


Figure 25: Population and bike fleet size by forecast temperature (log-log)

```
bike_population_weather <- dataset1 %>%
  inner_join(dataset2 %>% select(CITY, COUNTRY, POPULATION), by = c("CITY", "COUNTRY")) %>%
  inner_join(dataset3 %>% select(CITY, TEMP, HUMIDITY, WIND_SPEED), by = "CITY") %>%
  filter(!is.na(BICYCLES), !is.na(POPULATION), POPULATION > 0)
```

Data-quality flag — the notebook’s flagship chart is missing Seoul. This chained join produces **only three visible data points**, clustered between roughly 5.2M and 11.0M population. **Seoul (population 21,794,000; fleet 20,000) is not among them**, despite appearing individually in all three source tables — including cells 79/81/83 above, which join the same two tables (BIKE_SHARING_SYSTEMS and WEATHER_FORECAST) by CITY and clearly retain all three cities, Seoul included. R’s own many-to-many join warning fires twice during this cell’s execution, which is consistent with the same duplicate-row problem identified in Section 4.1 disrupting this particular chained join for Seoul specifically. **This should be re-run against de-duplicated source tables before being relied on** — as it stands, the notebook’s own description of this chart (“population from worldcities,” “bike fleet from bike_sharing_systems,” “forecast temperature from cities_weather_forecast” — genuinely combining three datasets) significantly overstates what’s actually being shown: two or three points, not a general relationship.

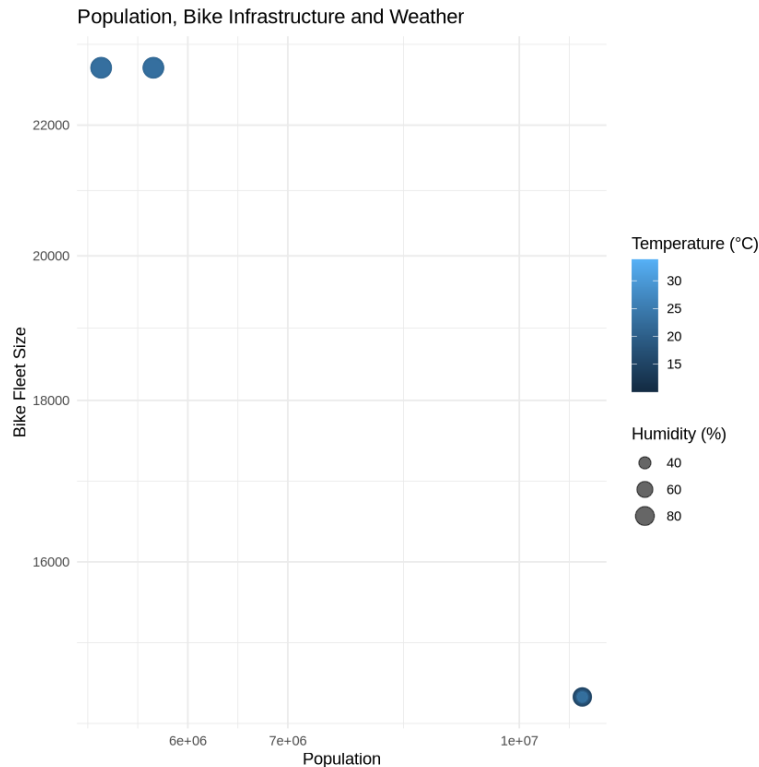


Figure 26: Population, bike infrastructure and weather, with humidity as point size

“Three datasets with weather represented by point size. I can also use humidity as an additional dimension.” Built from the same bike_population_weather data frame as above, so the **same Seoul gap persists** here.

6.3 Bicycles per 1,000 People vs. Temperature

“A cleaner three-dataset approach is to calculate bicycles per 1,000 people first.”

“This is arguably easier to interpret than raw fleet size because it accounts for city population.” Worth flagging carefully here: with Seoul absent, this LOESS curve is fit across **repeated 3-hourly readings from what is effectively just two cities** (Paris, at ~1.3 bicycles/1,000, and Suzhou, at ~4.0–4.4 bicycles/1,000) — each contributing dozens of near-duplicate y-values at slightly different x-values (temperature) across their forecast windows. **This is a textbook pseudo-replication setup:** the smoother has no real city-to-city sample size to work with, so the “rise, peak near 25°C, then fall” shape it produces is much more likely an artifact of Suzhou simply having warmer forecast readings than Paris during this window, rather than a genuine relationship between temperature and bike-share infrastructure density (which, being a fixed per-city quantity, cannot plausibly respond to weather at all). **This chart should not be used to support any claim about temperature affecting bike-share provision.**

6.4 Population, Infrastructure & Weather Together

“One visualization that ties together the three major city-level datasets.”

```
Population      -> processed_worldcities
Bicycles per 1000 -> bike_sharing_systems + worldcities
Temperature     -> cities_weather_forecast
```

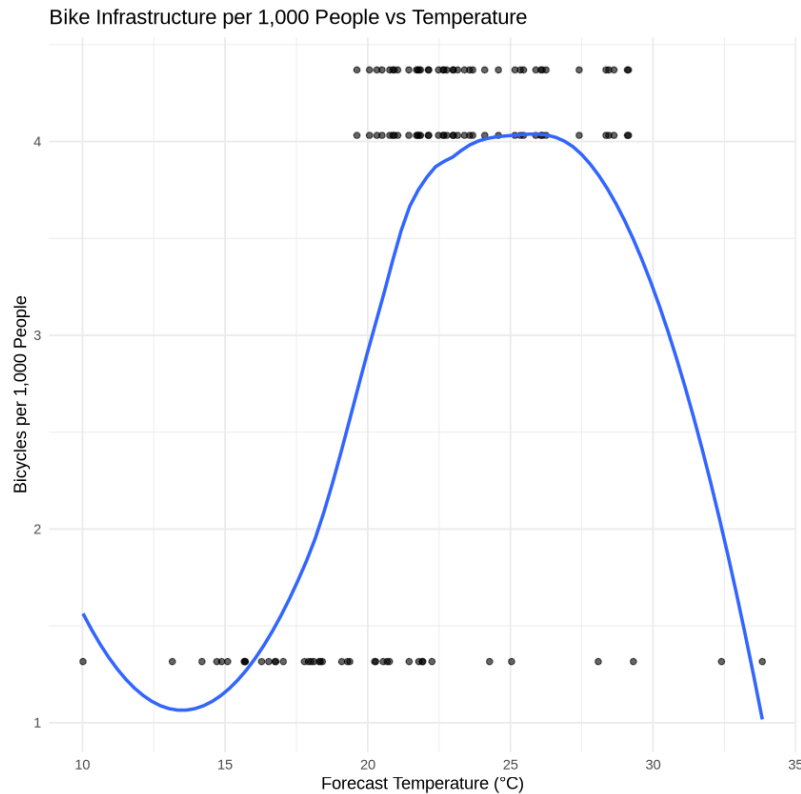


Figure 27: Bike infrastructure per 1,000 people vs forecast temperature, with LOESS smooth

Fleet size -> bike_sharing_systems (point size)

As the notebook’s closing chart, this is intended as the capstone visualization “tying together” all three city-level datasets — but it inherits the same gap as Section 6.2: **three points, no Seoul**. Given Seoul is the namesake city of the whole bike-sharing analysis, its absence from the notebook’s own summary chart is the single most consequential finding in this report and should be resolved before this chart (or the two before it) is shared or published as-is.

7. Consolidated Findings & Data-Quality Log

#	Area	Finding	Status
1	Seoul Bike-Sharing	<code>sum(is.na(dataset)) = 0</code> — table is fully clean at load time	Confirmed clean
2	Seoul Bike-Sharing	295 missing hours (288 from 12 fully-missing dates + 7 scattered single hours)	→ Data Science phase

#	Area	Finding	Status
3	Seoul Bike-Sharing	VISIBILITY right-censored at exactly 2,000 (median/3rd-Qu./max all near this ceiling)	Flagged for review
4	Seoul Bike-Sharing	All 8,465 rows already FUNCTIONING_DAY == "Yes" — explains absence of zero-count hours	Explained, not an issue
5	Seoul Bike-Sharing	Isolated large snowfall spike at year-end inflates Autumn's seasonal snowfall total	Flagged for review
6	Seoul Bike-Sharing	geom_smooth() LOESS fails on Rainfall/Snowfall charts due to near-zero variance	Plotting limitation, not a data issue
7	Bike-Sharing Systems	"City" #4 in the largest-fleets chart is a national description, not a city ("adjacent)	→ Data Science phase
8	Bike-Sharing Systems	Five cities tied at exactly 20,000 bicycles — likely placeholder/default values	Flagged for review
9	Cross-join (Cities × Systems)	Many-to-many join warning fires even on the corrected CITY+COUNTRY key	Confirms SQL report's duplicate-row cause
10	Cross-join (Cities × Systems)	Atlanta (~90/1,000) and Lanxi (~21/1,000) top the per-capita chart as join artifacts	Same root cause as SQL report §6.3
11	3-way join (Cities × Systems × Weather)	"Main three-dataset visualization" contains only 3 points; Seoul is missing	→ Data Science phase — high priority
12	3-way join	Bicycles-per-1,000-vs-temperature LOESS trend is likely a pseudo-replication artifact	Methodology flag, not a data issue

#	Area	Finding	Status
13	Weather Forecast	Isolated ~33 °C Paris temperature reading separated from the rest of the histogram	Cross-confirms SQL report §3.2

8. Recommendations for the Data Science Phase

1. **De-duplicate** `WORLD_CITIES` on `CITY + COUNTRY` before any further joins — this notebook independently confirms (via R’s own many-to-many warnings) that this is necessary even when `COUNTRY` is already included as a join key, not just when joining on city name alone.
2. **Investigate why the chained** `dataset1 JOIN dataset2 JOIN dataset3` **join drops Seoul** in Section 6.2–6.4 specifically, since Seoul survives each of the two joins individually elsewhere in the notebook. This is the most consequential single finding in this report, since it undermines the notebook’s headline three-dataset chart.
3. **Clean the CITY field in** `BIKE_SHARING_SYSTEMS` — remove or re-classify the “Towns and cities in the Netherlands...” record, and verify whether the five cities tied at exactly 20,000 bicycles are genuine or placeholder values.
4. **Re-derive the bicycles-per-1,000-vs-temperature relationship (Section 6.3) using proper city-level aggregation** (one observation per city, not per 3-hourly forecast reading) before drawing any conclusion about weather’s effect on infrastructure.
5. **Treat VISIBILITY = 2,000 as a possible measurement ceiling** rather than a true observed maximum when this field is used in further modeling.
6. Consider **separating out the isolated year-end snow event** (Section 2.7) from “typical” Autumn conditions if seasonal weather patterns are used as model features.

End of report.

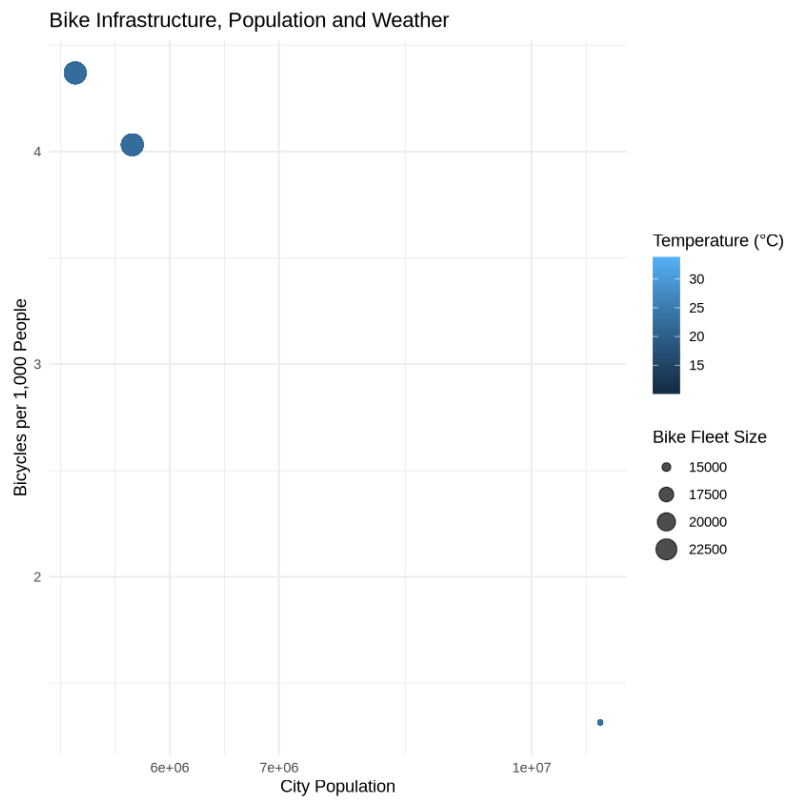


Figure 28: Bike infrastructure, population and weather, bubble chart